

2.2 The Gradient

Study Notes · Third Eye Labs
Season 2, episode 2

THE ONE SENTENCE

Ask 2.1's question once per knob, stack the answers into **one arrow** — and that arrow points straight **uphill**.

Two dials on a real photograph

A = washed-out photo (97,200 numbers)

y = the correct photo

w_0 = contrast w_1 = brightness

$\text{loss} = \text{mean}((w_0 \cdot A + w_1 - y)^2)$

BEFORE YOU START — if you write software but not machine learning

Episode 2.1 tuned one value. Real systems have many, and the interesting part is that you do not have to try them one at a time — you can work out all of their effects at once, and combine them into a single instruction.

the ML word	what it actually is, in your terms
gradient	A list of numbers, one per weight: how much the error moves if you nudge that weight. Two weights here, so two numbers. GPT-2 has 124 million.
vector	An array of numbers treated as one thing — and, usefully, as a direction. $[-0.118, -0.144]$ is both a pair of values and an arrow.
contour	A line along which the error does not change — like a contour on a map, where you walk without going up or down.
batch / mini-batch	Computing the error over a sample of the data rather than all of it, because all of it is too slow. Sampling, exactly as you would for a metric.

The punchline is a sampling result any engineer will recognise: 100 data points out of 97,200 give you an answer 1.1° off the true one. You almost never need the whole population.

1. Wiggle each dial separately

turn CONTRAST a hair $\rightarrow -0.11762$

turn BRIGHTNESS a hair $\rightarrow -0.14436$

stack: $g = [-0.11762, -0.14436]$

length $|g| = 0.18621 \leftarrow$ how steep it is here

finite differences, the exact formula and autograd all agree

2. Built one real pixel at a time

is	should be	off	$2 \times A \times \text{miss}$	running
0.700	0.922	-0.221	-0.3101	-0.3101
0.634	0.780	-0.147	-0.1860	-0.2480
0.730	0.973	-0.243	-0.3545	-0.2835
0.639	0.808	-0.168	-0.2154	-0.2665

...all 97,200 numbers average to -0.11762

3. The race — and why nothing can win

5,000 random directions vs the gradient.

Instantaneously: 0 of 5,000 beat it.

4. Perpendicular to the contour

$g \cdot \text{tangent} = 0.00e+00$

angle = 90.0000°

$$g \cdot \hat{d} = |g| \cos \theta$$

angle off	score
0°	+0.18621 ← the max
30°	+0.16127
60°	+0.09311
90°	0

Cosine never exceeds 1, and equals 1 in exactly one direction. That is the proof.

△ Over a finite step a few random directions can win — the surface curves away. Steepest means steepest instantaneously. That gap is 2.3.

6. What one gradient costs

GPT-2: 124,439,808 × 4 bytes
= 498 MB per step
× 300,000 steps = ~149 TB
computed and thrown away

Exactly, not approximately. Walk along a contour and the score does not change — so the direction of fastest change must be at a right angle to it.

5. Nobody computes the real one

pixels	angle off	downhill	cheaper
1	11.5°	65.7%	97,200×
10	3.8°	97.3%	9,720×
100	1.1°	100%	972×
1000	0.4°	100%	97×

Throw away 99.9% of your data and you still know which way to go. That is what *stochastic* / *mini-batch* / *batch mean*.

7. Advanced

- The gradient is local. It is only true where you stand; the surface curves away as soon as you move.
- Its length foreshadows 2.4: an arrow that shrinks toward zero on the way back through layers is the vanishing gradient.
- No gradient at all: the moment a model picks a token, a black-box API, a human preference. Those need a different toolbox entirely.

NEXT: you know which way. How far? → 2.3, the learning rate.